

ATFM: GPU results & caching tradeoffs

Evidence report | 28-29 September 2026

The serving integration works. Two public sessions completed faithfully, the tool-call API matrix passed 39 of 40 structural checks, and the larger session exposed cache-capacity pressure. We have not yet measured a performance advantage from ATFM's policy.

This report consolidates the local RTX 4060 compatibility checks, H100 compatibility check, public Weka replays, BFCL-derived API checks, failed attempts, and rental outcome. It then develops capacity and transfer calculations to guide the next experiments. Earlier CPU and simulation studies remain separate; see the research index [10].

Evidence	Result	What it establishes
CPU warming + real inference	22/22 chunks; 351/352 reused tokens	Supported actuator and cache integration
Two complete public roots	45/45 requests; no errors or truncation	Faithful completion of selected sessions
Expanded BFCL sample	39/40 call-shape checks	Tool-call API behavior on a small sample
Large isolated retry	92/119 retained records; timeout	Incomplete stress attempt; 19 L1 warnings
ATFM policy performance	Not evaluated	No policy speedup or optimum established

How to read the visuals

Measured means a retained experiment record or diagnostic. **Derived** means arithmetic using measured configuration. **Illustrative** means an explicitly assumed scenario, not an observed hardware result. Keeping these separate matters more than a single headline speedup.

Reading map

Pages 2-4 describe the setup, compatibility checks, and workload provenance. Pages 5-8 present latency, reuse, tool calls, and failures. Pages 9-13 explain memory, concurrency, chunks, bandwidth, and predictive warming. Pages 14-15 compare design choices and define the next experiment. Pages 16-17 provide reproducibility, accounting, references, and evidence links.

Main decision: first establish a complete large-session baseline and identify the critical path. Then compare bounded predictive warming against ordinary LMCACHE at identical resource budgets.

01 | What ran, and how it was measured

The public workload baseline uses one H100 NVL 94GB and Qwen/Qwen3-4B-Instruct-2507. ATFM control is disabled in these replays. The separate compatibility probe manually exercises ATFM's supported disk-to-CPU warming actuator. Sources: [1-4].

Component	Recorded configuration
GPU / driver	H100 NVL; 95,830 MiB reported; driver 580.159.04
Model + tokenizer revision	cdbee75f17c01a7cc42f958dc650907174af0554
Serving software	vLLM 0.30.0; LMCache 0.5.5; PyTorch 2.13.0+cu130
Load generator	AIPerf 0.13.0; reconstruction seed 7
Engine limits	Context 131,072; max sequences 2; GPU utilization setting 0.55
Execution / reuse	Eager; vLLM prefix caching off; LMCache external reuse on
CPU cache	24 GiB L1; 16-token chunks; LRU; 80% eviction watermark
Persistent cache	Filesystem L2 on MFS network storage, not local SSD
Storage allocation	50 GB container disk; 100 GB persistent workspace
Tool interface	Hermes parser; automatic tool choice; seed 7; temperature 0

Replay contract

Each selected Weka root runs once with `--num-sessions 1` and `--no-fixed-schedule`. Its child tree must finish. The runner preserves recorded token targets and end-to-start delays, with no synthetic input/output caps. `ignore_eos:true` requests the prescribed output length. This is closed-loop replay: replacement-model service times alter wall-clock arrival times.

TTFT is client-observed time to first token. Request latency covers the full response. Replay duration comes from AIPerf; harness elapsed time additionally includes setup. External-hit ratios use before/after vLLM Prometheus counter differences because per-response cached-token details were unavailable.

Experimental boundaries

The accepted short and sequential cases share a stack, in that order; they are not independent cold starts. Engine concurrency is capped at two sequences. GPU DCGM telemetry was unavailable. Sixteen-token chunks and eager mode were compatibility choices, not a tuned H100 throughput baseline. Hardware identity, commands, installed versions, source fingerprints, and raw logs are retained in [4].

02 | Compatibility: the actuator works

A 352-token synthetic prompt requests eight output tokens. After cold inference fills the cache, the probe clears its own CPU L1, verifies it is empty, submits an ATFM CPU-prefetch directive, waits for completion, and repeats inference. It checks reuse, text equality, occupancy, locks, and shutdown [2, 3].

Observation	RTX fresh repeat	RTX rebuilt env	H100 NVL
Cold external hits	0	0	0
Completed keys	22/22	22/22	22/22
Warm reused tokens	351/352	351/352	351/352
Text identical	Yes	Yes	Yes
CPU bytes after prefetch	40,370,176	40,370,176	40,370,176
Read / write locks	0 / 0	0 / 0	0 / 0
Cold HTTP elapsed	133.17 ms	131.78 ms	774.09 ms
Warm HTTP elapsed	101.84 ms	108.93 ms	244.74 ms

These are single diagnostic observations, not cross-hardware rankings, controlled speedups, or policy measurements. The model here is Qwen3-0.6B, not the 4B public-workload model. Its KV footprint must not be substituted into the later capacity analysis. vLLM recomputes the last prompt token, explaining 351 reported reused tokens.

Deployment failures that were resolved

The first H100 launch exceeded the six-minute readiness limit. A longer configurable deadline exposed a missing Ninja executable path. Adding the serving environment's bin directory fixed discovery. Native compilation then exposed mismatched CUDA compiler/header packages: nvcc, NVVM, and CRT were pinned to 13.0.88 alongside runtime 13.0.96. Toolkit lib64 and unversioned libcudart.so links completed the native setup.

What the success does and does not mean

The adapter confirms completed CPU warming, not merely HTTP acceptance. It rejects partial or unknown completion and reports unsupported GPU placement. CPU residency does not guarantee GPU residency or future retention. The test proves an action can work; it does not prove the forecasting board chooses useful actions under contention.

The RTX repeats used a 0.5 GiB CPU tier, 2,048-token engine context, and a rebuilt pinned environment. H100 compatibility passed at commit 3df2d53. Public workloads use a different model and much larger working sets [2, 3].

03 | Public workloads and fidelity

Three complete roots were selected from the SemiAnalysis Weka corpus [5]. They were copied without root truncation and retain per-root hashes. Selection favored context-compatible, manageable cases; this is a feasibility sample, not random sampling or an official AgentX evaluation.

Root	Requests	Source subagent groups	Max input + output	Recorded input / output
Short branch	21	1	54,926	818,752 / 3,949
Sequential	24	0	93,675	1,557,440 / 10,202
Multi-branch	119	3	118,677	9,906,624 / 37,635
Selected total	164	-	118,677	12,282,816 / 51,786

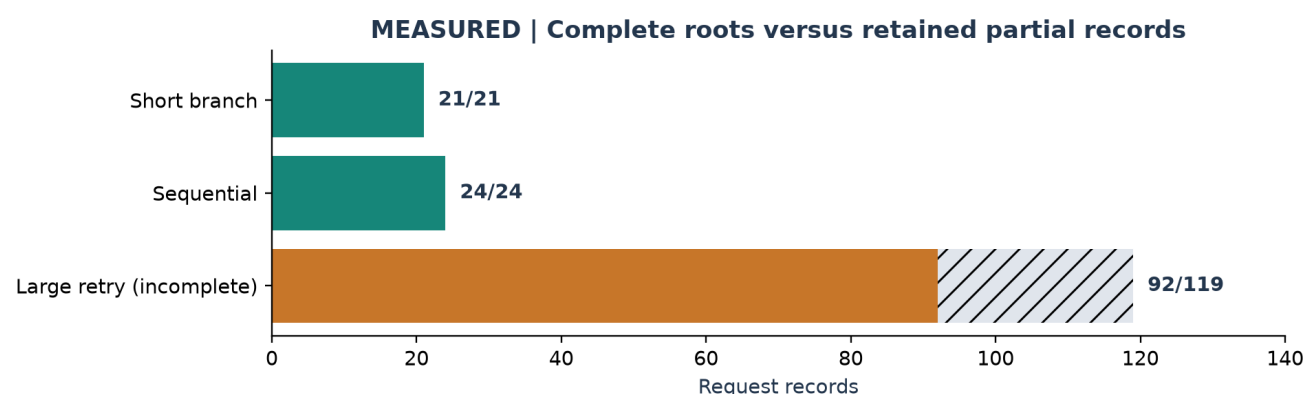


Figure 1 - Measured completion accounting. The large retry retained 92 completed request records; the other 27 are not evidenced as completed in that export. This does not classify them as 27 server errors. Only the first two bars represent accepted complete roots.

What is realistic, and what is reconstructed

Weka preserves a public workload's session structure, cache-hash relationships, sizes, and delays. Prompt text is reconstructed from cache hashes; original tool arguments and results are not present. Counts derive from cache blocks rather than the exact target tokenizer. Source model identities are replaced by one Qwen model. The selected roots have no `input_types` annotations, so they cannot validate tool-name-conditioned prediction.

BFCL provides actual questions and function schemas [6]. It complements this structural replay with tool-call API checks, but no external tools are executed. Neither dataset usage here demonstrates end-to-end agent task success.

04 | Complete-session latency results

The accepted seeded rerun at 4bcc012 finished 45 requests across two roots, with no reported errors or child truncations [1, 4]. Both produced the full requested server output-token totals. There is one run per root; reported percentiles describe those observations only.

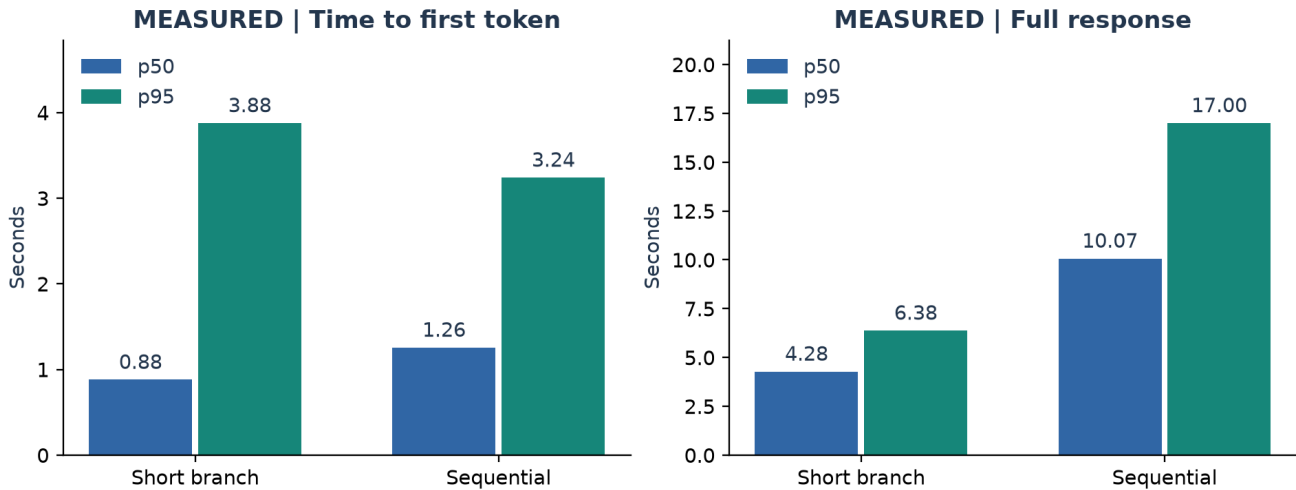


Figure 2 - Measured request latency and TTFT. Each panel uses its own scale. Bars compare different workloads, not competing policies. A p95 over 21 or 24 requests is unstable and is not a confidence interval.

Metric	Short branch	Sequential
Completed requests / children	21/21; 1/1	24/24; 0/0
TTFT p50 / p95	880.54 / 3,877.61 ms	1,255.72 / 3,240.73 ms
Full request p50 / p95	4,281.04 / 6,378.09 ms	10,066.22 / 17,003.67 ms
AIPerf replay duration	143.93 s	292.73 s
Harness elapsed	155.92 s	304.66 s
Output tokens, actual / target	3,949 / 3,949	10,202 / 10,202

Interpretation

Sequential requests have a larger median full-response latency, alongside more output tokens over the root. This is not evidence that cache reuse is ineffective: reuse primarily avoids prompt prefill, while new tokens still need decoding [7]. We did not isolate decode, transfer, queueing, or client scheduling time. Subtracting independently computed percentiles would not recover their phase contributions.

Replay duration includes recorded waits and session dependencies. Dividing tokens by that duration would describe this replay's delivered rate, not the H100's saturated inference capacity. A separate concurrency/load sweep is needed for a throughput claim.

05 | High reuse, finite residency

The external-hit counters show substantial reuse in both complete roots. These counters are token-weighted totals over requests, not the fraction of unique context retained, the share of requests with a full hit, or the share of time saved [1, 4].

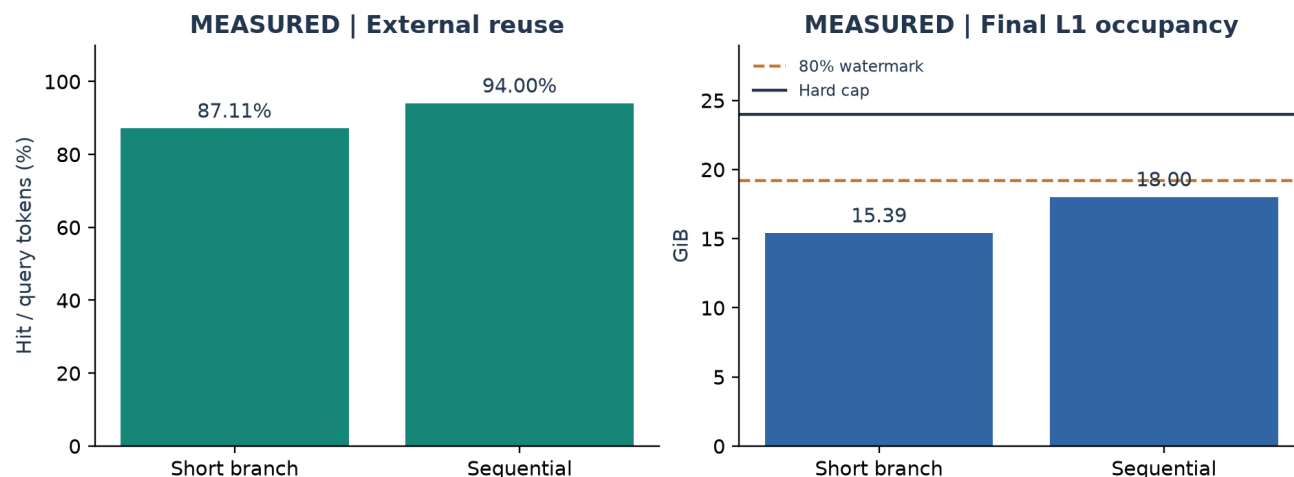


Figure 3 - Measured external token reuse and final CPU occupancy. The dashed occupancy line marks the configured 19.2 GiB eviction watermark; the solid line marks 24 GiB capacity. Final occupancy is not peak occupancy.

Counter / state	Short branch	Sequential
External hit tokens	714,560	1,466,560
External queried tokens	820,262	1,560,190
Hit / query	87.11%	94.00%
Final L1 bytes	16,529,227,776	19,329,712,128
Final objects	7,006	8,193
Final read / write locks	0 / 0	0 / 0

Why a high hit ratio can coexist with stalls

A repeated long prefix can dominate token counts, while a few expensive misses dominate latency. A hit in external storage still has to reach the GPU. Near-capacity allocation can require eviction or wait for locked/in-flight objects. The same prefix can be counted repeatedly across turns without representing that many distinct resident bytes.

The sequential root logged watermark-triggered evictions at 04:08:35, 04:09:47, and 04:11:46 UTC. Its 18.00 GiB final snapshot is compatible with earlier occupancy above the 19.2 GiB trigger. Zero locks at the end does not prove there was no earlier lock contention.

Implication: evaluate timely, useful reuse and end-to-end latency together. Maximizing an aggregate hit ratio alone is not a sufficient policy objective.

06 | Tool-call API results

The expanded BFCL-derived matrix at 1c33fa7 made 40 requests: the first ten rows of each of four pinned categories. All responses were saved. The aggregate verdict is false because one call-shape check failed [4, 6].

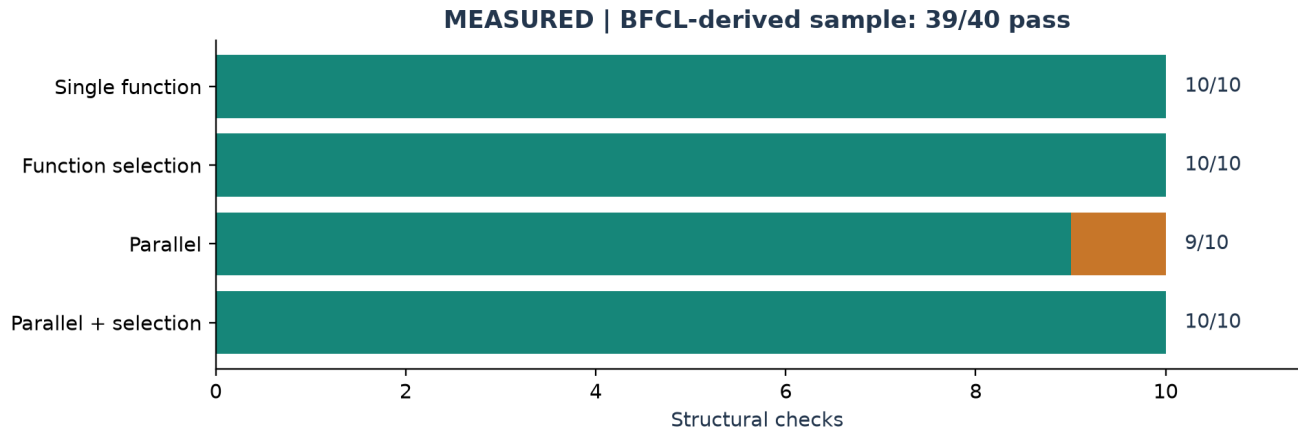


Figure 4 - Measured structural checks, not official BFCL accuracy. The sample passed 39/40 checks (97.5%). Convenience sampling and simplified scoring prevent interpreting this as benchmark-level model accuracy.

Validation and schema adaptation

Checks require an offered function name, arguments that parse as a JSON object, no length truncation, and at least two calls for parallel categories. Schema adaptation maps dict to object, float to number, and list to array; dotted function names become underscores and collisions are rejected. Questions remain unchanged. Requests use temperature zero, seed 7, and a 512-token output limit.

Initial nine-case prechecks (three per original category) passed in the retained successful precheck runs. They overlap the expanded sample and must not be added to its denominator as new independent examples. Tool-matrix timings are not presented as a latency benchmark because scratch cleanup overlapped startup.

The one discrepancy: parallel_9

The model emitted one `find_movie_showing` call with arrays containing two movies and times. The schema permits arrays, but our check requires separate parallel calls; the pinned official reference also contains two calls. This is a supported call-shape discrepancy, not proof that a real implementation would reject the batched request.

No tools were executed; argument meaning, real side effects, and task completion were not scored. The next tool-quality experiment should use the official scorer and a safe execution harness, separately from cache-performance comparisons.

07 | Incomplete attempts and root causes

The larger 119-request root has no accepted complete-run result. Two distinct problems occurred: persistent-storage exhaustion and, after cleanup, timeout with CPU-cache allocation pressure. These must not be conflated [1, 4].

Attempt	Observation	Treatment
Fixed-schedule replay	Nested timing rejected before inference	Invalid; use recorded closed-loop delays
Request-count-bounded replay	21 records but three truncated children	Rejected; bound complete root sessions
First session-bounded replay	21 complete; result handler argument missing	Raw export retained; seeded rerun accepted instead
Shared-stack large root	Disk quota exceeded at 04:20:38 UTC	Invalid partial; logs include null-filled tails
Isolated large retry	1,110 s supervisor expired; exit 124	Incomplete; no finalized whole-root profile

Storage failure

Approximately 73 GiB of generated L2 cache, 18 GiB of package downloads, and 7.6 GiB of model weights accumulated on a 100 GB workspace. These approximate GiB figures already exceed 100 decimal GB when combined, before other files. The shared filesystem's `df` displayed host-wide capacity rather than the Pod's quota. Evidence files sharing the same quota were damaged during flushing.

Generated KV and download caches were reclaimed; logs and responses were retained. The isolated retry had no new disk-quota errors. Future runs need explicit per-volume accounting, bounded L2 growth, and separate space reserved for evidence.

Retry outcome

The fresh-cache retry at `f441de1` retained 92 unique, non-cancelled records, each with server usage, totaling 30,433 output tokens versus 37,635 for the full root. The supervisor budget included startup. Its expiry does not isolate inference slowness from startup, recorded waits, or cache work.

The retry logged 19 L1 batch-allocation warnings. One requested 4,153 blocks of 2,359,296 bytes and lacked space for 1,301 blocks: about 9.12 GiB requested and 2.86 GiB short. Allocation warnings are not automatically HTTP failures. The final successful periodic snapshot preceded the final export (84 successes versus 92 retained records); it is not a final counter. Client event-loop delays of 28-55 ms occurred in the earlier large attempt, without proving they dominated latency.

08 | Memory: the first hard constraint

For a conventional uniform, uncompressed attention KV layout, tensor bytes per token are $b = 2 \times \text{layers} \times \text{KV_heads} \times \text{head_dimension} \times \text{bytes_per_element}$. The factor two represents keys and values. For this run, LMCache directly reports $b = 147,456$ bytes = 144 KiB. The calculations below use that observed value rather than assuming all models have this footprint [4].

For independent resident token sets, $M_{KV} = b \times U$, where U is the number of unique cached token positions. Add metadata, alignment, allocator overhead, in-flight buffers, and other processes when sizing the host. GiB means 2^{30} bytes.

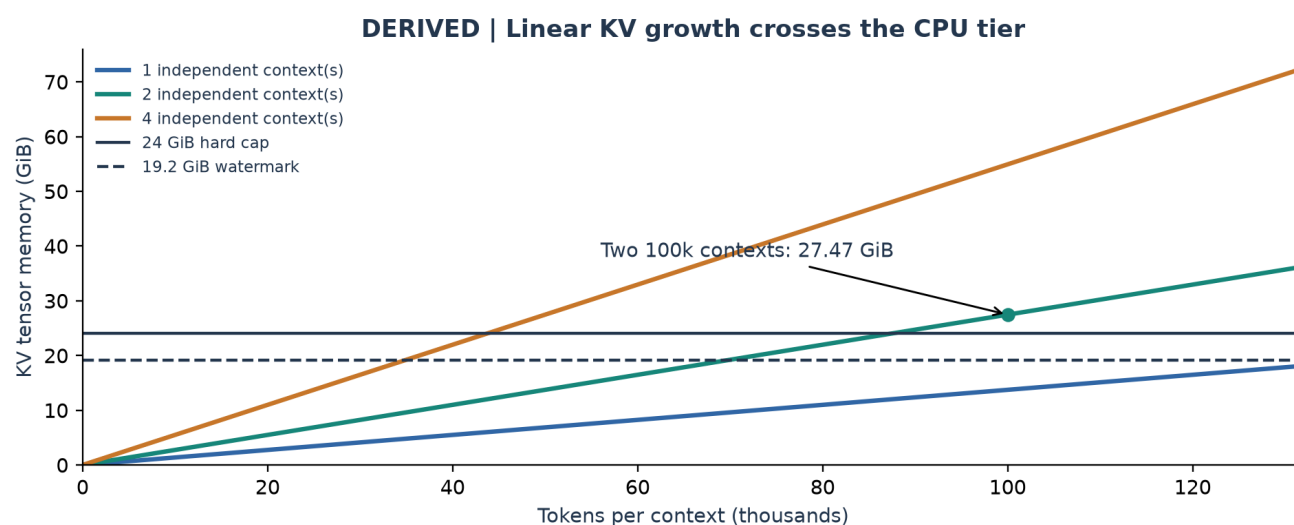


Figure 5 - Derived tensor demand. Lines assume independent contexts, no shared-prefix savings, and the measured 4B model layout. Crossings show capacity pressure, not a prediction of request failure. Eviction begins before the hard cap.

Scenario	KV tensors alone	Consequence
100,000 tokens, one context	13.73 GiB	Fits below 19.2 GiB watermark in isolation
100,000 tokens, two independent contexts	27.47 GiB	Exceeds the configured 24 GiB L1
131,072 tokens, one context	18.00 GiB	Little room below watermark for other objects
24 GiB hard-cap token equivalent	About 174,763 tokens	Idealized total, excluding overhead
19.2 GiB watermark equivalent	About 139,810 tokens	Eviction trigger, not a separate hard limit

More GPU VRAM does not automatically enlarge CPU L1. Likewise, a 55% GPU utilization setting is a memory-budget parameter, not measured GPU utilization or a CPU-cache limit. Engine max sequences bounds active engine work; retained historical prefixes and prefetch batches can create a larger CPU working set.

09 | Concurrency, sharing, and headroom

The relevant size is the union of resident cached objects, not simply active sessions multiplied by their full text lengths. Shared prefixes can reduce unique storage if the engine and cache identity permit deduplication. Branch suffixes, multiple models, and incompatible layouts can remove that advantage. These examples are sizing scenarios, not measured deduplication rates.

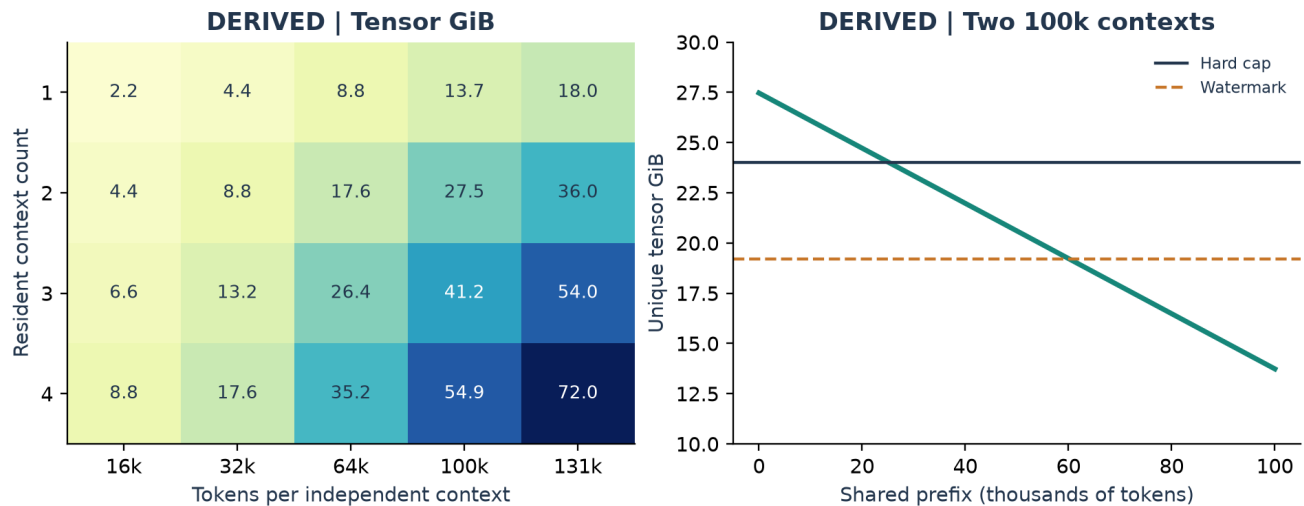


Figure 6 - Derived scenario analysis. Left: KV GiB versus independent context count and length. Right: two 100k-token contexts sharing P initial tokens require $b \times (200k - P)$ bytes, assuming each shared object is stored once in L1. The dashed line is the 19.2 GiB watermark.

Quantifying the headroom problem

For two 100k-token contexts, sharing more than approximately 25,237 tokens brings tensors below 24 GiB. Sharing more than approximately 60,190 tokens brings them below 19.2 GiB. Neither threshold includes metadata, unrelated cache objects, or temporary allocations. They are optimistic boundaries, not recommended operating targets.

A planning constraint is $\text{resident_bytes} + \text{in_flight_bytes} + \text{reserved_headroom} \leq \text{L1_capacity}$. If the implementation counts in-flight reservations inside resident allocation, avoid double-counting them. Measure allocator state and locked bytes explicitly; do not infer free allocatable space from an end-of-run object count.

Why more capacity is a diagnostic experiment

Moving from 24 to 48 GiB should reduce capacity pressure for the same unique working set, but is not guaranteed to reduce latency: transfer bandwidth, prefill, decode, or the client may dominate. A 64 GiB tier offers additional headroom but consumes host RAM needed for the OS, engine, staging, and other processes. Check actual available/pinnable RAM before increasing reservations.

The useful question is not whether every historical context fits forever. It is whether contexts needed soon can remain available without displacing more valuable ones or saturating the transfer path.

10 | Chunk size: objects versus granularity

The public runs use 16-token chunks. At 144 KiB per token, each full chunk contains 2.25 MiB of KV tensors. A 100k-token context therefore involves 6,250 chunks. Larger chunks reduce the number of objects and operations, but each allocation and transfer becomes larger. This tradeoff does not make total tensor bytes disappear.

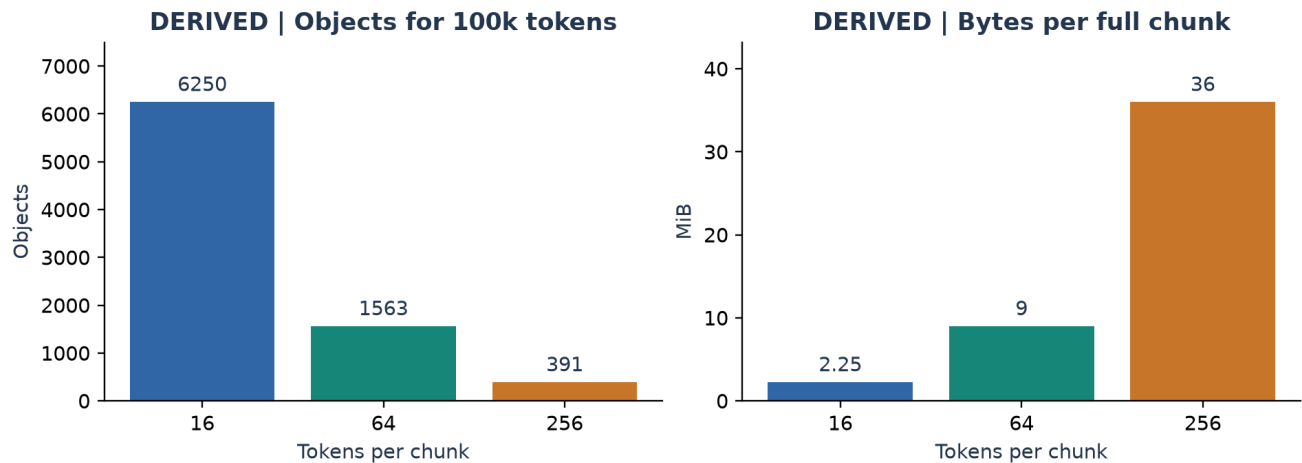


Figure 7 - Derived chunk sizing for 100,000 tokens. Object counts use `ceil(tokens / chunk_size)` as a capacity model. A last partial chunk may instead be omitted from reuse by a particular implementation. Actual behavior must be checked on the pinned connector.

Chunk tokens	Objects for 100k	Full chunk size	Max rounded tail per context
16	6,250	2.25 MiB	Less than 2.25 MiB
64	1,563	9 MiB	Less than 9 MiB
256	391	36 MiB	Less than 36 MiB

Expected benefits and costs

Larger chunks can amortize per-object hashing, metadata, queue dispatch, and filesystem operations. They also coarsen prefix reuse and eviction boundaries, enlarge reservation bursts, and may transfer bytes that are not subsequently useful. Smaller chunks allow finer retention and cancellation boundaries but can amplify fixed overhead at long contexts.

A simple work model is $T = \text{bytes} / \text{effective_bandwidth} + \text{object_count} \times \text{fixed_cost} + \text{queue_time}$. For fixed chunk size, object count and tensor movement grow linearly with context length. Metadata overhead can be reduced without changing the unavoidable amount of KV data needed by a request.

LMCache's upstream documentation discusses configurable chunks and persistent-tier behavior [8, 9]. Supported sizes, alignment, partial-chunk handling, and batched allocation behavior must be verified against our pinned 0.5.5 stack. Test 16, 64, and 256 separately after capacity is held constant; do not change chunk size and L1 size together and attribute the result to one factor.

11 | Moving KV versus recomputing it

An external-cache hit is not free. The relevant comparison is the wall-clock critical path for retrieving cached KV versus recomputing the same prefix. CPU warming can move L2 reads into an idle interval, but the supported actuator does not promise that data is already resident on the GPU.

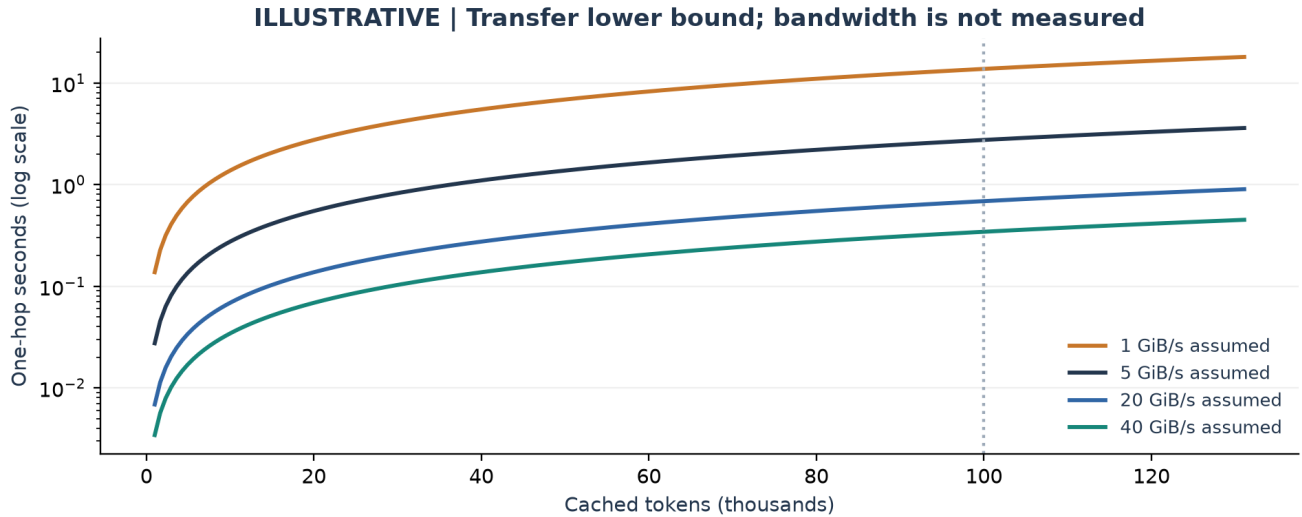


Figure 8 - Illustrative one-hop transfer lower bounds. Each line is $\text{time} = \text{KV_bytes} / \text{effective_bandwidth}$ for the measured 144 KiB/token layout. Bandwidth values are hypothetical sustained rates in GiB/s, not measurements or specifications of the rented Pod. The log scale shows the sensitivity across storage and host-transfer regimes.

For 100k tokens (13.73 GiB), a single hop takes at least 13.73 s at 1 GiB/s, 2.75 s at 5 GiB/s, 0.69 s at 20 GiB/s, or 0.34 s at 40 GiB/s. These bounds exclude lookup, allocation, dispatch, synchronization, and contention. Two serial hops add; overlapped pipelines approach the slower stage plus fill/drain overhead. Shared bandwidth limits concurrent transfers.

Break-even test

Define $T_{\text{recompute}}$ as measured remaining prefill time without the reusable KV, and T_{retrieve} as retrieval plus exposed synchronization on the critical path. Retrieval helps that request when $T_{\text{retrieve}} < T_{\text{recompute}}$, subject to effects on other requests. Neither side has been measured in isolation here.

Dense-attention prefill has a quadratic attention-work term in context length, while KV size and byte transfer grow linearly. Decode attends over the existing context for each new token. These asymptotic statements do not supply a numerical break-even: kernels, batching, model dimensions, precision, and bandwidth determine constants. Efficient attention implementations need not materialize a quadratic-size score matrix.

Python micro-optimization cannot remove a tens-of-GiB working set. Equally, this evidence does not rule out CPU overhead: thousands of chunks and observed event-loop delays justify profiling. Measure per-object CPU cost and critical-path waits before considering a native-language rewrite.

12 | Predictive warming: timing matters

ATFM's potential value is to use an agent's tool-running interval to load context before the next LLM request. A useful prediction needs both the right context and the right time. Loading too late exposes transfer delay; loading too early ties up capacity and risks eviction before use.

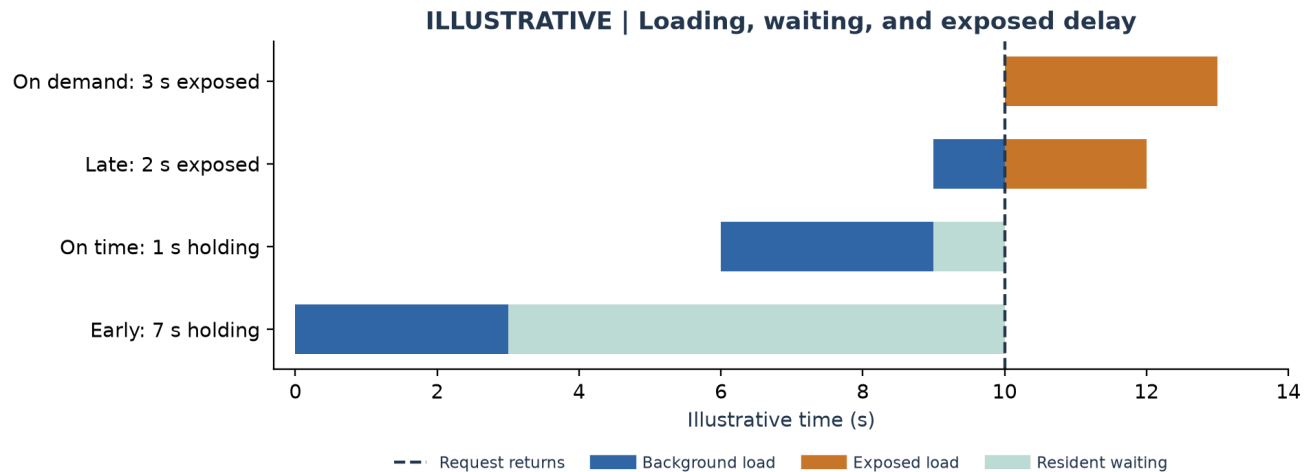


Figure 9 - Illustrative timing only. Assume a request returns at $t=10$ s and L2-to-L1 loading takes 3 s. An on-demand load exposes all 3 s; a load ending at $t=12$ exposes 2 s; one ending at $t=9$ hides that stage with a 1 s residency wait. A load ending at $t=3$ holds memory for 7 s and might be evicted. GPU transfer and inference are omitted and still have to occur.

A decision rule to test, not a proven optimum

Let p be the probability of reuse before eviction, S the request latency saved if the warm data is useful, D the expected displacement penalty to other requests, and C the transfer/contention cost. In a common objective unit, an action is attractive when $p \times S > C + D$. Memory and bandwidth budgets remain hard constraints, even if this expected gain is positive. Costs can be expressed as latency penalties or explicit weighted resource prices, but unlike units must not be added directly.

A practical controller should bound pending jobs, reserve room for demand reads, prioritize likely near-term returns, and track completed useful bytes rather than accepted API calls. Calibration and timeliness matter: an accurate eventual-return prediction can still warm too soon. Completion must be followed by a check that data remained available until demand.

These public runs did not enable the policy and lack selected-root tool-type signals. The compatibility probe proves CPU warming works; it does not validate the rule above. A policy comparison must include failed/wasted prefetches, eviction of useful data, background bandwidth consumption, and overhead from observation and control.

13 | Design choices and their tradeoffs

There is no hardware-independent best configuration. Separate capacity feasibility, transfer efficiency, and prediction quality so that one cannot hide a weakness in another.

Choice	Potential value	Cost / risk	What to measure
Increase L1 to 48 or 64 GiB	Fewer evictions and failed batch reservations	More pinned/host RAM; may not improve the critical path	Peak allocated/locked bytes; warnings; TTFT
Bound L2, reserve evidence space	Avoid quota failures; keep durable diagnostics	More L2 evictions and possible recompute	Actual volume bytes; write errors; recoverability
Use faster/local L2	Shorter cold external-cache reads	Rental/storage cost; topology changes	Cold-read bandwidth and p95; OS-cache state
Enlarge chunks	Fewer objects, operations, and file accesses	Coarser reuse/eviction; larger bursts	CPU time/object; throughput; useful bytes
Batch and pipeline loads	Amortize overhead; overlap transfer stages	More in-flight memory; demand interference	Stage overlap; reservation peaks; tail latency
Reduce concurrent active contexts	Keep working set within capacity	More admission queueing and lower possible throughput	End-to-end latency including queue; completions/s
Predictive CPU warming	Hide L2 read time during tool waits	Wrong/early loads; displacement; shared bandwidth	Useful-on-time prefetch rate; net latency benefit
KV compression / lower precision	Fewer bytes to retain and move	Conversion cost; compatibility and quality concerns	Quality parity; conversion time; net bytes/time
CUDA graphs / engine tuning	Lower execution overhead	Different memory budget and compatibility constraints	Controlled eager-versus-graph comparison

What seems highest value now

First, make the large root finish with intact evidence and known storage headroom. Second, vary CPU capacity while holding everything else fixed. Third, isolate read/transfer/prefill costs and tune chunking. Only then compare the predictive controller at matched budgets. This ordering is an experimental recommendation, not a claim that capacity is the dominant latency bottleneck.

A better policy cannot keep 27.47 GiB of independent tensors simultaneously inside 24 GiB. It can choose which bytes to keep, when to load them, and which requests to admit. Quantization, sharing, compression, or more capacity change the physical footprint; scheduling changes when that footprint is needed.

14 | The next experiment, designed to decide

Use the same complete 119-request root, fixed reconstruction seed, model revision, and delays. Keep the measurement horizon long enough for the full root, or predeclare a fixed observation window and explicitly change the claim. Do not rescue a result by silently truncating the workload.

Stage	Controlled comparison	Decision it informs
A: valid baseline	Fresh cache; 24 GiB; sufficient L2/evidence quota	Can the complete root finish under this configuration?
B: capacity	24 vs 48 GiB; optionally 64 GiB after host checks	Does more L1 remove warnings and improve latency?
C: retrieval path	Cold recompute; L2 present/L1 empty; verified warm L1	Where is useful reuse faster than recomputation?
D: granularity	16 vs 64 vs 256 tokens at fixed L1	Is per-object overhead material?
E: policy	Ordinary LMCACHE; observation-only ATFM; bounded warming	Does prediction add value after its costs?

Instrument the critical path

For each request, align monotonic timestamps for client send, server receipt, queue admission, lookup, L2 read, allocation waits, CPU-to-GPU copy, remaining prefill, first token, and completion. Capture bytes and object counts alongside durations. Stage durations can overlap: report exposed waits and concurrency, not an unqualified sum. Record CPU profiles and event-loop lag separately from GPU utilization.

Sample allocated/reserved/locked L1, in-flight bytes, evictions, load failures, L2 footprint, demand versus speculative traffic, and exact filesystem quota. A warm filesystem page cache is not a cold disk read; predeclare how that state is controlled. Retain logs on capacity-reserved storage.

Acceptance and comparison rules

Require all expected requests and child branches, full output targets, no truncation, complete exports, correct cleanup, and no quota errors. Run multiple independent repetitions with randomized condition order and fresh cache state where appropriate. Three repetitions can be a feasibility floor, not a precision guarantee. Use paired whole-run comparisons and intervals that respect correlated requests; do not treat every token as an independent sample.

Choose primary outcomes before running: complete-session duration, request TTFT distribution, and end-to-end latency including holds. Report useful/wasted prefetch bytes and hardware cost alongside latency. Validate any selected configuration on held-out roots. An offline clairvoyant policy may provide a bound only if it obeys the same capacities and transfer costs and is clearly labeled as using future information.

15 | Reproducibility, fixes, and accounting

The archived artifacts are the authority for experiment claims. This PDF adds analysis and visualizations; it does not replace the raw records. Sources [1-4] link exact configurations, command lines, request exports, counters, source fingerprints, and checksums.

Revision	Role
3df2d53	Passing H100 compatibility check
676bb9f	First full short-root run; reconstruction seed not yet pinned
4bcc012	Accepted seeded short + sequential results; handler fix
1c33fa7	Expanded 40-case BFCL-derived tool matrix
f441de1	Isolated selected-root retry with regression coverage
739f902	Final retained stress outcome and shutdown documentation

The session stop condition was changed from request count to one complete root. The replay seed was pinned to 7. Result handling regained its expected-count argument. Tool-check failures now affect the overall verdict. A selected-case option allows an isolated root without repeating earlier cases. These runner changes leave core ATFM policy logic unchanged.

Final recorded regression validation was **603 passed, 3 skipped**, with six existing warnings. This is the code validation recorded with the experiments; generating this report does not claim a new GPU run or a new full-suite result.

Rental outcome

Both Pods were confirmed EXITED. The original allocation ran from 02:36:41.930 to 03:26:25 UTC; the replacement from 03:40:16.821 until stop was confirmed by 04:47:58 UTC on 29 September. Created-to-confirmed-stop windows total 7,044.249 seconds, or 117.4 minutes, below the approved aggregate 120 minutes.

At the quoted \$3.19/hour, that is approximately \$6.24 compute. This is an estimate, not an invoice; storage, tax, and provider billing adjustments are excluded. Both persistent volumes remained allocated and chargeable at the last recorded check. The original Pod's restart failed because its host had no free GPU.

Retained versus excluded data

Three public-workload archives include the invalid attempts, complete seeded cases, tool matrix, and isolated retry. Their local hashes matched remote copies before shutdown. Model weights and bulk KV tensor files are excluded. The repository retains 24 checksummed curated files in the public H100 evidence bundle, plus separate local/H100 compatibility evidence. Null-filled tails from the quota failure remain original evidence, not silently repaired data.

16 | Sources and evidence index

The repository links below refer to the committed experiment record at 739f902. Live upstream documentation was consulted on 29 September 2026 for conceptual interpretation; it can describe capabilities newer than the pinned runtime. It is not evidence that a feature was enabled in these runs.

[1] [Public GPU workloads: detailed measured report](#). Accepted sessions, incomplete attempts, diagnostics, and shutdown.

[2] [H100 compatibility report](#). Native build fixes and completed warming/reuse.

[3] [Local RTX 4060 compatibility report](#). Fresh and rebuilt environment runs, actuator scope, limitations.

[4] [Public H100 evidence bundle](#) and [reproduction guide](#). Machine-readable profiles, manifests, tool responses, retry records, full archives, and SHA-256 index.

[5] [SemiAnalysis Weka trace corpus](#). Exact selected root IDs and hashes are in the evidence bundle's `selection.json`; the source file SHA-256 is

29b6a19e751ff5230771519aab755f80a0f43a4ba9cf96b72d3a6a437ec99276.

[6] [BFCL pinned dataset tree](#). Four V4 categories; first ten rows each. Saved `parallel_9-reference.json` preserves the expected two-call reference and its source hash.

[7] [vLLM automatic prefix caching](#). Conceptual distinction between reused prefill and new-token decoding; this documentation version is not the serving version used in the experiment.

[8] [LMCache MP configuration](#). Reference for configurable chunks, memory managers, and bounded in-flight prefetch concepts. Verify supported settings against the pinned release.

[9] [LMCache persistent L2 storage](#). Storage-tier and eviction concepts; no upstream benchmark numbers are imported here.

[10] [ATFM research index](#). Earlier CPU, forecasting, simulation, and local service results, intentionally kept distinct from this real-GPU campaign.

Evidence archive fingerprints

`public-initial-evidence.tar.gz:`

255ace7eebc45c92ce2d54c5bdd2e094022c2853f7675c5c9f5b6396409af7fe

`public-tool-evidence.tar.gz:`

9368e2373ddac4fecdd696b9319db4f79fc1d4f61bafadf35da035fb8202e8e08

`public-multi-retry-evidence.tar.gz:`

46c7c963c747fc2f37f676d7c74db07171d2f10dfdea887e2cacb465c4d39931

All nine figures in this report are generated from retained observations or explicitly labeled calculations. No figure represents an unperformed policy trial.